

Problem Statements

1. You are developing a multi-channel notification system for a large enterprise similar to Amazon Web Services.

The system must support Email, SMS, and Push notifications.

Creating separate service objects repeatedly leads to memory overhead and inconsistent configurations.

Design a solution where a Factory Pattern dynamically selects the notification type at runtime, while each notification service is implemented as a Singleton to ensure only one instance exists per type.

2. You are developing a custom travel itinerary generation system similar to Make-MyTrip, where users can create personalized travel plans by selecting various components such as hotel type, transport mode, meal plan, and sightseeing options. Since each itinerary may have different combinations of these components, the object construction process becomes complex and requires step-by-step configuration.

To address this, you are required to implement the Builder Design Pattern. The system must define a Product class (Itinerary) representing the final travel plan. An abstract Builder class (ItineraryBuilder) should declare methods for building different parts of the itinerary. Multiple Concrete Builder classes (e.g., LuxuryItineraryBuilder, BudgetItineraryBuilder) must implement these methods to construct specific types of itineraries.

A Director class (TravelDirector) should be responsible for orchestrating the construction process by invoking builder methods in a specific sequence.

Additionally, a Singleton Configuration Manager must be implemented to provide global settings such as default budget limits, travel policies, and discount rules. All builders must access this shared instance instead of creating new configuration objects.

Requirements:

- Design Product, Builder, Concrete Builder, and Director classes
- Implement a Singleton Configuration Manager
- Demonstrate construction of at least two types of itineraries
- Ensure no direct object creation of configuration using new